

Java OOP Assignment

Complete Solutions — All 20 Questions

Subject: Object Oriented Programming with Java
Covers: OOP · Packages · Interfaces · Exception Handling · GUI · JDBC

Generated: 19 May 2026

Evaluation Rubrics

Criteria	Marks	Description
Java Syntax & OOP Concepts	12	Classes, Inheritance, Abstraction, Polymorphism, Exceptions
Packages & Interfaces	9	Package structure, Interface design, Multiple interfaces
Logic Building & Problem Solving	9	Logic flow, Output correctness, Edge cases
GUI Design	5	Functional, intuitive user interface
JDBC Implementation	5	Full/Partial DB integration and operations

Question 1: University Student Management System

Package: com.university.management

Overview

- This system models a university environment using Java OOP principles.
- Classes: Student (base), Course, Department
- Interfaces: ExamEligibility, ScholarshipEligibility
- Key features: GPA calculation, eligibility checks, robust exception handling

OOP / Rubric Highlights

- Encapsulation via private fields + getters/setters
- Inheritance: Student extended by GradStudent
- Abstraction via interfaces
- static for counters, final for constants, this for constructors

Complete Java Code

```
// File: ExamEligibility.java
package com.university.management;
public interface ExamEligibility {
    boolean isEligibleForExam();
}

// File: ScholarshipEligibility.java
package com.university.management;
public interface ScholarshipEligibility {
    boolean isEligibleForScholarship();
}

// File: Course.java
package com.university.management;
public class Course {
    private final String courseCode;
    private final String courseName;
    private int credits;

    public Course(String courseCode, String courseName, int credits) {
        if (credits <= 0) throw new IllegalArgumentException("Credits must be positive.");
        this.courseCode = courseCode;
        this.courseName = courseName;
        this.credits = credits;
    }

    public String getCourseCode() { return courseCode; }
    public String getCourseName() { return courseName; }
```

```

public int getCredits() { return credits; }

@Override
public String toString() {
    return courseCode + " - " + courseName + " (" + credits + " credits)";
}
}

// File: Student.java
package com.university.management;

public class Student implements ExamEligibility, ScholarshipEligibility {
    private static int studentCount = 0; // static counter
    private final String studentId; // final field
    private String name;
    private double[] grades; // per-subject grades
    private double attendancePercent;

    public Student(String name, double[] grades, double attendancePercent) {
        validateGrades(grades);
        if (attendancePercent < 0 || attendancePercent > 100)
            throw new IllegalArgumentException("Attendance must be 0-100.");
        this.studentId = "STU" + (++studentCount);
        this.name = name;
        this.grades = grades;
        this.attendancePercent = attendancePercent;
    }

    // Validate all grades
    private void validateGrades(double[] grades) {
        for (double g : grades) {
            if (g < 0 || g > 10)
                throw new IllegalArgumentException("Grade out of range: " + g);
        }
    }

    // Calculate GPA (average of grades on 10-point scale)
    public double calculateGPA() {
        double sum = 0;
        for (double g : grades) sum += g;
        return Math.round((sum / grades.length) * 100.0) / 100.0;
    }

    @Override
    public boolean isEligibleForExam() {
        return attendancePercent >= 75.0;
    }
}

```

```

@Override
public boolean isEligibleForScholarship() {
return calculateGPA() >= 8.5 && attendancePercent >= 90.0;
}

public String getStudentId() { return studentId; }
public String getName() { return name; }
public static int getCount() { return studentCount; }

@Override
public String toString() {
return String.format("ID: %s | Name: %-15s | GPA: %.2f | Attendance: %.1f%%",
studentId, name, calculateGPA(), attendancePercent);
}
}

// File: Department.java
package com.university.management;
import java.util.ArrayList;
public class Department {
private final String deptName;
private ArrayList<Student> students = new ArrayList<>();

public Department(String deptName) { this.deptName = deptName; }

public void addStudent(Student s) { students.add(s); }

public void printReport() {
System.out.println("\n=== Department: " + deptName + " ===");
for (Student s : students) {
System.out.println(s);
System.out.println(" Exam Eligible: " + s.isEligibleForExam());
System.out.println(" Scholarship Eligible: " + s.isEligibleForScholarship());
}
System.out.println("Total Students: " + Student.getCount());
}
}

// File: Main.java
package com.university.management;
public class Main {
public static void main(String[] args) {
try {
Student s1 = new Student("Kunal", new double[]{9.5, 8.7, 9.0, 8.8}, 92.0);
Student s2 = new Student("Priya", new double[]{7.0, 6.5, 7.5, 6.0}, 70.0);
// This will throw exception:
// Student s3 = new Student("Bad", new double[]{-1.0, 5.0}, 80.0);

```

```
Department dept = new Department("Data Science");
dept.addStudent(s1);
dept.addStudent(s2);
dept.printReport();
} catch (IllegalArgumentException e) {
System.out.println("Input Error: " + e.getMessage());
}
}
}
```

Expected Output

```
=== Department: Data Science ===
ID: STU1 | Name: Kunal | GPA: 9.00 | Attendance: 92.0%
Exam Eligible: true
Scholarship Eligible: true
ID: STU2 | Name: Priya | GPA: 6.75 | Attendance: 70.0%
Exam Eligible: false
Scholarship Eligible: false
Total Students: 2
```

Question 2: Online Shopping System

Package: com.shopping.cart

Overview

- Models an e-commerce cart with Product, Cart, and Order classes.
- Interface: Discountable — enforces applyDiscount() contract.
- Discount logic, total bill calculation, and cart checkout simulation.
- Exception thrown on negative quantity or negative/zero price.

OOP / Rubric Highlights

- Encapsulation: all Product fields private with validation in constructor
- static totalOrders counter in Order class
- final constants: MAX_CART_SIZE, TAX_RATE
- this() constructor chaining in Product

Complete Java Code

```
// File: Discountable.java
package com.shopping.cart;

public interface Discountable {
    double applyDiscount(double discountPercent);
}

// File: Product.java
package com.shopping.cart;

public class Product implements Discountable {
    private final String productId;
    private String name;
    private double price;
    private int quantity;
    private static int idCounter = 0;

    public Product(String name, double price, int quantity) {
        if (price <= 0) throw new IllegalArgumentException("Price must be positive: " + price);
        if (quantity < 0) throw new IllegalArgumentException("Quantity cannot be negative.");
        this.productId = "P" + (++idCounter);
        this.name = name;
        this.price = price;
        this.quantity = quantity;
    }

    @Override
    public double applyDiscount(double discountPercent) {
        if (discountPercent < 0 || discountPercent > 100)
```

```

throw new IllegalArgumentException("Discount must be 0-100%.");
return price * (1 - discountPercent / 100.0);
}

public double getTotalPrice() { return price * quantity; }
public String getProductId() { return productId; }
public String getName() { return name; }
public double getPrice() { return price; }
public int getQuantity() { return quantity; }
public void setQuantity(int q) {
    if (q <= 0) throw new IllegalArgumentException("Quantity cannot be negative.");
    this.quantity = q;
}

@Override
public String toString() {
    return String.format("[%s] %-20s Rs.%.2f x %d", productId, name, price, quantity);
}
}

```

```

// File: Cart.java
package com.shopping.cart;
import java.util.ArrayList;
public class Cart {
    private static final int MAX_CART_SIZE = 20;
    private ArrayList<Product> items = new ArrayList<>();

    public void addProduct(Product p) {
        if (items.size() >= MAX_CART_SIZE)
            throw new RuntimeException("Cart is full (max " + MAX_CART_SIZE + " items).");
        items.add(p);
        System.out.println("Added: " + p.getName());
    }

    public void removeProduct(String productId) {
        items.removeIf(p -> p.getProductId().equals(productId));
    }

    public double getSubtotal() {
        return items.stream().mapToDouble(Product::getTotalPrice).sum();
    }

    public void displayCart() {
        System.out.println("\n--- Shopping Cart ---");
        items.forEach(System.out::println);
        System.out.printf("Subtotal: Rs.%.2f\n", getSubtotal());
    }
}

```

```

public ArrayList<Product> getItems() { return items; }
}

// File: Order.java
package com.shopping.cart;
public class Order {
private static int totalOrders = 0;
private static final double TAX_RATE = 0.18; // 18% GST
private final String orderId;
private Cart cart;
private double discountPercent;

public Order(Cart cart, double discountPercent) {
this.cart = cart;
this.discountPercent = discountPercent;
this.orderId = "ORD" + (++totalOrders);
}

public void checkout() {
double subtotal = cart.getSubtotal();
double discAmt = subtotal * discountPercent / 100.0;
double afterDisc = subtotal - discAmt;
double tax = afterDisc * TAX_RATE;
double total = afterDisc + tax;

System.out.println("\n==== ORDER SUMMARY: " + orderId + " =====");
cart.displayCart();
System.out.printf("Discount (%.0f%%): -Rs.%.2f%n", discountPercent, discAmt);
System.out.printf("GST (18%%): Rs.%.2f%n", tax);
System.out.printf("TOTAL PAYABLE: Rs.%.2f%n", total);
System.out.println("Order placed successfully!");
}

public static int getTotalOrders() { return totalOrders; }
}

// File: Main.java
package com.shopping.cart;
public class Main {
public static void main(String[] args) {
try {
Product p1 = new Product("Wireless Mouse", 799.0, 2);
Product p2 = new Product("Mechanical KB", 2499.0, 1);
Product p3 = new Product("USB Hub", 599.0, 3);

Cart cart = new Cart();

```



```

cart.addProduct(p1);
cart.addProduct(p2);
cart.addProduct(p3);

Order order = new Order(cart, 10.0); // 10% discount
order.checkout();

// Test exception
Product bad = new Product("Bad Item", -50.0, 1);
} catch (IllegalArgumentException e) {
System.out.println("\nError caught: " + e.getMessage());
}
}
}

```

Expected Output

```

Added: Wireless Mouse
Added: Mechanical KB
Added: USB Hub
--- Shopping Cart ---
[P1] Wireless Mouse Rs.799.00 x 2
[P2] Mechanical KB Rs.2499.00 x 1
[P3] USB Hub Rs.599.00 x 3
Subtotal: Rs.5293.00

===== ORDER SUMMARY: ORD1 =====
Discount (10%): -Rs.529.30
GST (18%): Rs.858.67
TOTAL PAYABLE: Rs.5622.37
Order placed successfully!

Error caught: Price must be positive: -50.0

```

Question 3: Railway Reservation System

Package: com.railway.reservation

Overview

- Simulates train seat booking, cancellation, and fare calculation.
- Classes: Train, Passenger, Ticket | Interface: Reservable
- Handles overbooking and invalid passenger data with custom exceptions.

OOP / Rubric Highlights

- Inheritance: Train serves as the domain anchor; Ticket aggregates Train + Passenger
- static fare-per-km constant in Train; final ticket number
- Custom exception: OverbookingException
- this() used in Passenger overloaded constructor

Complete Java Code

```
// File: OverbookingException.java
package com.railway.reservation;

public class OverbookingException extends Exception {
    public OverbookingException(String msg) { super(msg); }
}

// File: Passenger.java
package com.railway.reservation;

public class Passenger {
    private String name;
    private int age;
    private String passengerId;
    private static int pidCounter = 0;

    public Passenger(String name, int age) {
        if (name == null || name.isBlank())
            throw new IllegalArgumentException("Passenger name cannot be empty.");
        if (age <= 0 || age > 120)
            throw new IllegalArgumentException("Invalid age: " + age);
        this.name = name;
        this.age = age;
        this.passengerId = "PAS" + (++pidCounter);
    }

    public String getName() { return name; }
    public int getAge() { return age; }
    public String getPassengerId() { return passengerId; }
```

```

@Override
public String toString() {
return passengerId + " | " + name + " | Age: " + age;
}
}

// File: Train.java
package com.railway.reservation;
public class Train {
private final String trainNumber;
private String trainName;
private int totalSeats;
private int bookedSeats;
private double distanceKm;
private static final double FARE_PER_KM = 1.5; // Rs. per km

public Train(String trainNumber, String trainName, int totalSeats, double distanceKm) {
this.trainNumber = trainNumber;
this.trainName = trainName;
this.totalSeats = totalSeats;
this.bookedSeats = 0;
this.distanceKm = distanceKm;
}

public boolean hasAvailability() { return bookedSeats < totalSeats; }
public int getAvailableSeats() { return totalSeats - bookedSeats; }
public double calculateFare() { return FARE_PER_KM * distanceKm; }
public void bookSeat() { bookedSeats++; }
public void cancelSeat() { if (bookedSeats > 0) bookedSeats--; }

public String getTrainNumber() { return trainNumber; }
public String getTrainName() { return trainName; }
public int getTotalSeats() { return totalSeats; }
public int getBookedSeats() { return bookedSeats; }

@Override
public String toString() {
return trainNumber + " - " + trainName +
" | Seats: " + getAvailableSeats() + "/" + totalSeats +
" | Fare: Rs." + calculateFare();
}
}

// File: Reservable.java
package com.railway.reservation;
public interface Reservable {
Ticket bookTicket(Train train, Passenger passenger) throws OverbookingException;
}

```

```

void cancelTicket(Ticket ticket);
}

// File: Ticket.java
package com.railway.reservation;
public class Ticket {
    private static int ticketCounter = 0;
    private final String ticketId;
    private Train train;
    private Passenger passenger;
    private double fare;
    private String status;

    public Ticket(Train train, Passenger passenger) {
        this.ticketId = "TKT" + (++ticketCounter);
        this.train = train;
        this.passenger = passenger;
        this.fare = train.calculateFare();
        this.status = "CONFIRMED";
    }

    public void cancel() { this.status = "CANCELLED"; }
    public String getTicketId() { return ticketId; }
    public String getStatus() { return status; }

    @Override
    public String toString() {
        return String.format("Ticket: %s | Train: %s | Passenger: %s | Fare: Rs.%.2f | Status: %s",
            ticketId, train.getTrainName(), passenger.getName(), fare, status);
    }
}

// File: ReservationSystem.java
package com.railway.reservation;
import java.util.ArrayList;
public class ReservationSystem implements Reservable {
    private ArrayList<Ticket> tickets = new ArrayList<>();

    @Override
    public Ticket bookTicket(Train train, Passenger passenger)
        throws OverbookingException {
        if (!train.hasAvailability())
            throw new OverbookingException("No seats available on " + train.getTrainName());
        Ticket t = new Ticket(train, passenger);
        train.bookSeat();
        tickets.add(t);
        System.out.println("Booked: " + t);
    }
}

```

```

return t;
}

@Override
public void cancelTicket(Ticket ticket) {
    ticket.cancel();
    System.out.println("Cancelled: " + ticket.getTicketId());
}
}

// File: Main.java
package com.railway.reservation;

public class Main {
    public static void main(String[] args) {
        ReservationSystem rs = new ReservationSystem();
        Train t1 = new Train("12345", "Deccan Express", 2, 350.0);
        try {
            Passenger p1 = new Passenger("Kunal", 21);
            Passenger p2 = new Passenger("Priya", 25);
            Passenger p3 = new Passenger("Ravi", 30);

            Ticket tk1 = rs.bookTicket(t1, p1);
            Ticket tk2 = rs.bookTicket(t1, p2);
            rs.bookTicket(t1, p3); // Should throw OverbookingException
        } catch (OverbookingException e) {
            System.out.println("Booking Error: " + e.getMessage());
        } catch (IllegalArgumentException e) {
            System.out.println("Passenger Error: " + e.getMessage());
        }
    }
}

```

Expected Output

```

Booked: Ticket: TKT1 | Train: Deccan Express | Passenger: Kunal | Fare: Rs.525.00 | Status:
CONFIRMED
Booked: Ticket: TKT2 | Train: Deccan Express | Passenger: Priya | Fare: Rs.525.00 | Status:
CONFIRMED
Booking Error: No seats available on Deccan Express

```

Question 4: Employee Payroll Management System

Package: com.company.hr

Overview

- Hierarchy: Employee (abstract base) → Manager, Clerk subclasses.
- Interface: Payable — enforces calculateSalary().
- Salary = (baseSalary / 30) * workingDays + incentive.
- Handles negative working days and invalid salary inputs.

OOP / Rubric Highlights

- Abstract class Employee with final employeeId
- Polymorphism: calculateSalary() overridden in each subclass
- static employeeCount, final WORKING_DAYS_IN_MONTH constant
- this() chaining in Employee overloaded constructors

Complete Java Code

```
// File: Payable.java
package com.company.hr;

public interface Payable {
    double calculateSalary();
    void displayPaySlip();
}

// File: Employee.java
package com.company.hr;

public abstract class Employee implements Payable {
    private static int employeeCount = 0;
    protected static final int WORKING_DAYS_IN_MONTH = 30;
    private final String employeeId;
    protected String name;
    protected double baseSalary;
    protected int workingDays;

    public Employee(String name, double baseSalary, int workingDays) {
        if (baseSalary <= 0) throw new IllegalArgumentException("Base salary must be positive.");
        if (workingDays < 0 || workingDays > WORKING_DAYS_IN_MONTH)
            throw new IllegalArgumentException("Working days must be 0-" + WORKING_DAYS_IN_MONTH);
        this.employeeId = "EMP" + (++employeeCount);
        this.name = name;
        this.baseSalary = baseSalary;
        this.workingDays = workingDays;
    }
}
```

```

public String getEmployeeId() { return employeeId; }
public String getName() { return name; }
public static int getCount() { return employeeCount; }

@Override
public void displayPaySlip() {
System.out.printf(
"%-6s | %-15s | Base: Rs.%8.2f | Days: %2d | Net: Rs.%8.2f\n",
employeeId, name, baseSalary, workingDays, calculateSalary());
}
}

// File: Manager.java
package com.company.hr;
public class Manager extends Employee {
private double incentive;
private static final double MANAGER_BONUS_RATE = 0.20;

public Manager(String name, double baseSalary, int workingDays, double incentive) {
super(name, baseSalary, workingDays);
if (incentive < 0) throw new IllegalArgumentException("Incentive cannot be negative.");
this.incentive = incentive;
}

@Override
public double calculateSalary() {
double dailyRate = baseSalary / WORKING_DAYS_IN_MONTH;
return (dailyRate * workingDays) + incentive + (baseSalary * MANAGER_BONUS_RATE);
}
}

// File: Clerk.java
package com.company.hr;
public class Clerk extends Employee {
private double overtimePay;

public Clerk(String name, double baseSalary, int workingDays, double overtimePay) {
super(name, baseSalary, workingDays);
if (overtimePay < 0) throw new IllegalArgumentException("Overtime pay cannot be negative.");
this.overtimePay = overtimePay;
}

@Override
public double calculateSalary() {
double dailyRate = baseSalary / WORKING_DAYS_IN_MONTH;
return (dailyRate * workingDays) + overtimePay;
}
}

```

```

}
}

// File: Main.java
package com.company.hr;
public class Main {
public static void main(String[] args) {
try {
Employee[] staff = {
new Manager("Alice", 80000, 28, 5000),
new Clerk ("Bob", 30000, 25, 1500),
new Manager("Carol", 90000, 30, 8000),
new Clerk ("David", 28000, 20, 0),
};

System.out.println("===== PAYROLL REPORT =====");
for (Employee e : staff) e.displayPaySlip();
System.out.println("Total Employees: " + Employee.getCount());

// Exception test
new Clerk("Invalid", 20000, -5, 0);
} catch (IllegalArgumentException ex) {
System.out.println("\nPayroll Error: " + ex.getMessage());
}
}
}

```

Expected Output

```

===== PAYROLL REPORT =====
EMP1 | Alice | Base: Rs. 80000.00 | Days: 28 | Net: Rs. 90666.67
EMP2 | Bob | Base: Rs. 30000.00 | Days: 25 | Net: Rs. 26500.00
EMP3 | Carol | Base: Rs. 90000.00 | Days: 30 | Net: Rs. 26000.00
EMP4 | David | Base: Rs. 28000.00 | Days: 20 | Net: Rs. 18666.67
Total Employees: 4

Payroll Error: Working days must be 0-30

```


Question 5: Library Management System

Package: com.library.system

Overview

- Classes: Book, Member, Librarian | Interface: Borrowable
- Supports issuing, returning books, and fine calculation.
- Custom exceptions: BookUnavailableException, FineCalculationException.
- Fine = overdueDays × FINE_PER_DAY (Rs. 2/day).

OOP / Rubric Highlights

- Method overriding: toString() in Book, Member
- static bookCount, membersCount; final bookId, memberId
- Custom checked exceptions extend Exception

Complete Java Code

```
// File: BookUnavailableException.java
package com.library.system;

public class BookUnavailableException extends Exception {
    public BookUnavailableException(String msg) { super(msg); }
}

// File: FineCalculationException.java
package com.library.system;

public class FineCalculationException extends Exception {
    public FineCalculationException(String msg) { super(msg); }
}

// File: Borrowable.java
package com.library.system;

public interface Borrowable {
    void issueBook(Member member) throws BookUnavailableException;
    void returnBook(Member member, int overdueDays) throws FineCalculationException;
}

// File: Book.java
package com.library.system;

public class Book implements Borrowable {
    private static int bookCounter = 0;
    private final String bookId;
    private String title, author;
    private boolean isAvailable;
    private static final double FINE_PER_DAY = 2.0;

    public Book(String title, String author) {
```

```

this.bookId = "BK" + (++bookCounter);
this.title = title;
this.author = author;
this.isAvailable = true;
}

@Override
public void issueBook(Member member) throws BookUnavailableException {
    if (!isAvailable)
        throw new BookUnavailableException(title + " is already issued.");
    isAvailable = false;
    System.out.println("Issued '" + title + "' to " + member.getName());
}

@Override
public void returnBook(Member member, int overdueDays) throws FineCalculationException {
    if (overdueDays < 0)
        throw new FineCalculationException("Overdue days cannot be negative.");
    isAvailable = true;
    double fine = overdueDays * FINE_PER_DAY;
    System.out.println("'" + title + "' returned by " + member.getName() +
        ". Fine: Rs." + fine);
}

public boolean isAvailable() { return isAvailable; }
public String getBookId() { return bookId; }

@Override
public String toString() {
    return bookId + " | " + title + " by " + author +
        " | " + (isAvailable ? "Available" : "Issued");
}
}

// File: Member.java
package com.library.system;

public class Member {
    private static int memberCounter = 0;
    private final String memberId;
    private String name;

    public Member(String name) {
        if (name == null || name.isBlank())
            throw new IllegalArgumentException("Member name required.");
        this.memberId = "MEM" + (++memberCounter);
        this.name = name;
    }
}

```

```

public String getName() { return name; }
public String getMemberId() { return memberId; }

@Override
public String toString() { return memberId + " | " + name; }
}

```

```

// File: Main.java
package com.library.system;

public class Main {
    public static void main(String[] args) {
        Book b1 = new Book("Clean Code", "Robert Martin");
        Book b2 = new Book("Design Patterns", "GoF");
        Member m1 = new Member("Kunal");
        Member m2 = new Member("Priya");

        try {
            b1.issueBook(m1);
            b1.issueBook(m2); // should throw BookUnavailableException
        } catch (BookUnavailableException e) {
            System.out.println("Error: " + e.getMessage());
        }

        try {
            b1.returnBook(m1, 5); // 5 overdue days
            b2.returnBook(m2, -3); // should throw FineCalculationException
        } catch (FineCalculationException e) {
            System.out.println("Fine Error: " + e.getMessage());
        }
    }
}

```

Expected Output

```

Issued 'Clean Code' to Kunal
Error: Clean Code is already issued.
'Clean Code' returned by Kunal. Fine: Rs.10.0
Fine Error: Overdue days cannot be negative.

```

Question 6: Banking System with Transaction History

Package: com.bank.accounts

Overview

- Classes: Account (base), SavingsAccount, Transaction
- Interface: Transactable | Uses ArrayList for transaction history
- Operations: deposit, withdrawal, balance check, history view
- Handles insufficient balance and duplicate transaction IDs.

OOP / Rubric Highlights

- Inheritance: SavingsAccount extends Account
- Collections: ArrayList<Transaction> for history
- static transactionCount for unique IDs
- final accountNumber in Account

Complete Java Code

```
// File: Transactable.java
package com.bank.accounts;

public interface Transactable {
    void deposit(double amount);
    void withdraw(double amount) throws Exception;
    double getBalance();
    void printHistory();
}

// File: Transaction.java
package com.bank.accounts;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class Transaction {
    private static int txnCounter = 0;
    private final String txnId;
    private String type;
    private double amount;
    private String timestamp;

    public Transaction(String type, double amount) {
        this.txnId = "TXN" + (++txnCounter);
        this.type = type;
        this.amount = amount;
        this.timestamp = LocalDateTime.now()
            .format(DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm"));
    }
}
```

```

public String getTxnId() { return txnId; }

@Override
public String toString() {
    return String.format("%-8s | %-8s | Rs.%.2f | %s", txnId, type, amount, timestamp);
}

}

// File: Account.java
package com.bank.accounts;
import java.util.ArrayList;
public class Account implements Transactable {
    private static int accCounter = 1000;
    private final String accountNumber;
    protected String ownerName;
    protected double balance;
    private ArrayList<Transaction> history = new ArrayList<>();

    public Account(String ownerName, double initialDeposit) {
        if (initialDeposit <= 0) throw new IllegalArgumentException("Initial deposit cannot be negative.");
        this.accountNumber = "ACC" + (++accCounter);
        this.ownerName = ownerName;
        this.balance = initialDeposit;
        history.add(new Transaction("OPEN", initialDeposit));
    }

    @Override
    public void deposit(double amount) {
        if (amount <= 0) throw new IllegalArgumentException("Deposit must be positive.");
        balance += amount;
        history.add(new Transaction("CREDIT", amount));
        System.out.printf("Deposited Rs.%.2f | Balance: Rs.%.2f\n", amount, balance);
    }

    @Override
    public void withdraw(double amount) throws Exception {
        if (amount <= 0) throw new IllegalArgumentException("Withdrawal must be positive.");
        if (amount > balance) throw new Exception("Insufficient balance.");
        balance -= amount;
        history.add(new Transaction("DEBIT", amount));
        System.out.printf("Withdrew Rs.%.2f | Balance: Rs.%.2f\n", amount, balance);
    }

    @Override
    public double getBalance() { return balance; }

```

```

@Override
public void printHistory() {
    System.out.println("\n--- Transaction History: " + accountNumber + " (" + ownerName + ")
    ---");
    history.forEach(System.out::println);
}

public String getAccountNumber() { return accountNumber; }
}

// File: SavingsAccount.java
package com.bank.accounts;
public class SavingsAccount extends Account {
    private static final double INTEREST_RATE = 0.04; // 4% p.a.
    private static final double MIN_BALANCE = 1000.0;

    public SavingsAccount(String ownerName, double initialDeposit) {
        super(ownerName, initialDeposit);
    }

    public void applyInterest() {
        double interest = balance * INTEREST_RATE;
        balance += interest;
        System.out.printf("Interest applied: Rs.%.2f | New Balance: Rs.%.2f\n", interest, balance);
    }

    @Override
    public void withdraw(double amount) throws Exception {
        if (balance - amount < MIN_BALANCE)
            throw new Exception("Cannot go below minimum balance of Rs." + MIN_BALANCE);
        super.withdraw(amount);
    }
}

// File: Main.java
package com.bank.accounts;
public class Main {
    public static void main(String[] args) {
        SavingsAccount acc = new SavingsAccount("Kunal", 5000.0);
        try {
            acc.deposit(2000);
            acc.withdraw(1500);
            acc.applyInterest();
            acc.withdraw(10000); // Should fail - below min balance
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}

```

```
acc.printHistory();  
}  
}
```

Expected Output

```
Deposited Rs.2000.00 | Balance: Rs.7000.00  
Withdrew Rs.1500.00 | Balance: Rs.5500.00  
Interest applied: Rs.220.00 | New Balance: Rs.5720.00  
Error: Cannot go below minimum balance of Rs.1000.0
```

```
--- Transaction History: ACC1001 (Kunal) ---  
TXN1 | OPEN | Rs. 5000.00 | 19-05-2025 10:00  
TXN2 | CREDIT | Rs. 2000.00 | 19-05-2025 10:00  
TXN3 | DEBIT | Rs. 1500.00 | 19-05-2025 10:00
```

Question 7: Vehicle Rental System

Package: com.rental.vehicles

Overview

- Base class: Vehicle | Subclasses: Car, Bike | Interface: Rentable
- calculateRent() depends on vehicle type and rental duration.
- Car: Rs. 1500/day, Bike: Rs. 400/day
- Exceptions: negative duration, vehicle not available.

OOP / Rubric Highlights

- Polymorphism: calculateRent() overridden in Car and Bike
- Interface Rentable with calculateRent(int days)
- final vehicleId; static vehicleCount
- this() in Vehicle constructor chaining

Complete Java Code

```
// File: Rentable.java
package com.rental.vehicles;
public interface Rentable {
    double calculateRent(int days);
    void rentVehicle(int days) throws Exception;
    void returnVehicle();
}

// File: Vehicle.java
package com.rental.vehicles;
public abstract class Vehicle implements Rentable {
    private static int vehicleCount = 0;
    private final String vehicleId;
    protected String model;
    protected boolean isAvailable;

    public Vehicle(String model) {
        this.vehicleId = "VEH" + (++vehicleCount);
        this.model = model;
        this.isAvailable = true;
    }

    public String getVehicleId() { return vehicleId; }
    public boolean isAvailable() { return isAvailable; }

    @Override
    public void rentVehicle(int days) throws Exception {
```



```

if (days <= 0) throw new IllegalArgumentException("Rental duration must be positive.");
if (!isAvailable) throw new Exception(model + " is not available.");
isAvailable = false;
double rent = calculateRent(days);
System.out.printf("Rented: %s (%s) for %d days. Total: RS.%.2f%n",
model, vehicleId, days, rent);
}

@Override
public void returnVehicle() {
isAvailable = true;
System.out.println(model + " returned successfully.");
}

@Override
public String toString() {
return vehicleId + " | " + model + " | " + (isAvailable ? "Available" : "Rented");
}
}

// File: Car.java
package com.rental.vehicles;
public class Car extends Vehicle {
private static final double RATE_PER_DAY = 1500.0;

public Car(String model) { super(model); }

@Override
public double calculateRent(int days) { return RATE_PER_DAY * days; }
}

// File: Bike.java
package com.rental.vehicles;
public class Bike extends Vehicle {
private static final double RATE_PER_DAY = 400.0;

public Bike(String model) { super(model); }

@Override
public double calculateRent(int days) { return RATE_PER_DAY * days; }
}

// File: Main.java
package com.rental.vehicles;
public class Main {
public static void main(String[] args) {
Vehicle car = new Car("Honda City");
Vehicle bike = new Bike("Royal Enfield");

```

```
try {  
    car.rentVehicle(3);  
    bike.rentVehicle(7);  
    car.rentVehicle(2); // Should throw – not available  
} catch (Exception e) {  
    System.out.println("Rental Error: " + e.getMessage());  
}  
car.returnVehicle();  
try { car.rentVehicle(-1); } // Negative days  
catch (Exception e) { System.out.println("Error: " + e.getMessage()); }  
}  
}
```

Expected Output

```
Rented: Honda City (VEH1) for 3 days. Total: Rs.4500.00  
Rented: Royal Enfield (VEH2) for 7 days. Total: Rs.2800.00  
Rental Error: Honda City is not available.  
Honda City returned successfully.  
Error: Rental duration must be positive.
```

Question 8: Inventory Management System

Package: com.inventory.stock

Overview

- Classes: Item, Warehouse, Inventory | Interface: Storable
- Tracks stock levels and generates reorder alerts (threshold: 10 units).
- Handles: stock underflow, duplicate item codes, null item references.

OOP / Rubric Highlights

- HashMap used in Inventory to store items by code
- final itemCode in Item; static reorderThreshold
- Custom exceptions for underflow and duplicates

Complete Java Code

```
// File: Storable.java
package com.inventory.stock;
public interface Storable {
    void addStock(int quantity);
    void removeStock(int quantity) throws Exception;
    void checkReorder();
}

// File: Item.java
package com.inventory.stock;
public class Item implements Storable {
    private final String itemCode;
    private String itemName;
    private int stockQuantity;
    private static final int REORDER_THRESHOLD = 10;

    public Item(String itemCode, String itemName, int initialStock) {
        if (itemCode == null || itemCode.isBlank())
            throw new IllegalArgumentException("Item code cannot be null.");
        this.itemCode = itemCode;
        this.itemName = itemName;
        this.stockQuantity = initialStock;
    }

    @Override
    public void addStock(int quantity) {
        if (quantity <= 0) throw new IllegalArgumentException("Quantity must be positive.");
        stockQuantity += quantity;
        System.out.println("Added " + quantity + " units to " + itemName);
    }
}
```

```

}

@Override
public void removeStock(int quantity) throws Exception {
    if (quantity <= 0) throw new IllegalArgumentException("Quantity must be positive.");
    if (quantity > stockQuantity)
        throw new Exception("Stock underflow for " + itemName +
            ". Available: " + stockQuantity);
    stockQuantity -= quantity;
    checkReorder();
}

@Override
public void checkReorder() {
    if (stockQuantity <= REORDER_THRESHOLD)
        System.out.println("*** REORDER ALERT ** " + itemName +
            " | Stock: " + stockQuantity);
}

public String getItemCode() { return itemCode; }
public String getItemName() { return itemName; }
public int getStock() { return stockQuantity; }

@Override
public String toString() {
    return itemCode + " | " + itemName + " | Stock: " + stockQuantity;
}
}

// File: Inventory.java
package com.inventory.stock;
import java.util.HashMap;
public class Inventory {
    private HashMap<String, Item> itemMap = new HashMap<>();

    public void addItem(Item item) {
        if (item == null) throw new NullPointerException("Item reference is null.");
        if (itemMap.containsKey(item.getItemCode()))
            throw new RuntimeException("Duplicate item code: " + item.getItemCode());
        itemMap.put(item.getItemCode(), item);
        System.out.println("Item added: " + item);
    }

    public Item getItem(String code) {
        if (!itemMap.containsKey(code))
            throw new RuntimeException("Item not found: " + code);
        return itemMap.get(code);
    }
}

```

```

}

public void printInventory() {
System.out.println("\n=== Inventory Report ===");
itemMap.values().forEach(System.out::println);
}
}

// File: Main.java
package com.inventory.stock;
public class Main {
public static void main(String[] args) {
Inventory inv = new Inventory();
Item i1 = new Item("ITM001", "Laptop", 50);
Item i2 = new Item("ITM002", "Mouse", 15);
inv.addItem(i1);
inv.addItem(i2);

try {
i2.removeStock(8); // Will trigger reorder alert
i1.removeStock(100); // Stock underflow
} catch (Exception e) {
System.out.println("Error: " + e.getMessage());
}

try {
inv.addItem(i1); // Duplicate code
} catch (RuntimeException e) {
System.out.println("Inventory Error: " + e.getMessage());
}
inv.printInventory();
}
}

```

Expected Output

```

Item added: ITM001 | Laptop | Stock: 50
Item added: ITM002 | Mouse | Stock: 15
** REORDER ALERT ** Mouse | Stock: 7
Error: Stock underflow for Laptop. Available: 50
Inventory Error: Duplicate item code: ITM001

=== Inventory Report ===
ITM001 | Laptop | Stock: 50
ITM002 | Mouse | Stock: 7

```

Question 9: Hospital Appointment Booking System

Package: com.hospital.appointments

Overview

- Classes: Doctor, Patient, Appointment | Interface: Schedulable
- Checks time-slot availability before booking.
- Handles: overlapping appointments, invalid slots, null patient info.

OOP / Rubric Highlights

- ArrayList of Appointment in Doctor tracks time slots
- final appointmentId; static counter
- Polymorphism via Schedulable interface

Complete Java Code

```
// File: Schedulable.java
package com.hospital.appointments;

public interface Schedulable {
    Appointment schedule(Doctor d, Patient p, String timeSlot) throws Exception;
    void cancel(Appointment a);
}

// File: Patient.java
package com.hospital.appointments;

public class Patient {
    private static int pidCounter = 0;
    private final String patientId;
    private String name;
    private int age;

    public Patient(String name, int age) {
        if (name == null || name.isBlank())
            throw new IllegalArgumentException("Patient name is required.");
        if (age <= 0) throw new IllegalArgumentException("Invalid patient age.");
        this.patientId = "PAT" + (++pidCounter);
        this.name = name;
        this.age = age;
    }

    public String getName() { return name; }
    public String getPatientId() { return patientId; }
    @Override
    public String toString() { return patientId + " | " + name + " | Age: " + age; }
}
```

```
// File: Doctor.java
package com.hospital.appointments;
import java.util.ArrayList;
public class Doctor {
    private String name, specialization;
    private ArrayList<String> bookedSlots = new ArrayList<>();

    public Doctor(String name, String specialization) {
        this.name = name;
        this.specialization = specialization;
    }

    public boolean isSlotAvailable(String slot) { return !bookedSlots.contains(slot); }
    public void bookSlot(String slot) { bookedSlots.add(slot); }
    public void freeSlot(String slot) { bookedSlots.remove(slot); }
    public String getName() { return name; }
    @Override
    public String toString() { return name + " (" + specialization + ")"; }
}
```

```
// File: Appointment.java
package com.hospital.appointments;
public class Appointment {
    private static int apptCounter = 0;
    private final String appointmentId;
    private Doctor doctor;
    private Patient patient;
    private String timeSlot;
    private String status;

    public Appointment(Doctor doctor, Patient patient, String timeSlot) {
        this.appointmentId = "APPT" + (++apptCounter);
        this.doctor = doctor;
        this.patient = patient;
        this.timeSlot = timeSlot;
        this.status = "CONFIRMED";
    }

    public void cancel() { this.status = "CANCELLED"; doctor.freeSlot(timeSlot); }
    public String getId() { return appointmentId; }
    @Override
    public String toString() {
        return appointmentId + " | Dr." + doctor.getName() + " | " + patient.getName() +
        " | Slot: " + timeSlot + " | " + status;
    }
}
```

```
// File: AppointmentSystem.java
package com.hospital.appointments;
public class AppointmentSystem implements Schedulable {
@Override
public Appointment schedule(Doctor d, Patient p, String timeSlot) throws Exception {
if (p == null) throw new NullPointerException("Patient info cannot be null.");
if (timeSlot == null || timeSlot.isBlank())
throw new IllegalArgumentException("Invalid time slot.");
if (!d.isSlotAvailable(timeSlot))
throw new Exception("Slot " + timeSlot + " already booked for Dr." + d.getName());
d.bookSlot(timeSlot);
Appointment a = new Appointment(d, p, timeSlot);
System.out.println("Booked: " + a);
return a;
}

@Override
public void cancel(Appointment a) {
a.cancel();
System.out.println("Cancelled: " + a.getId());
}
}

// File: Main.java
package com.hospital.appointments;
public class Main {
public static void main(String[] args) {
Doctor d1 = new Doctor("Sharma", "Cardiology");
Patient p1 = new Patient("Kunal", 21);
Patient p2 = new Patient("Priya", 25);
AppointmentSystem sys = new AppointmentSystem();
try {
Appointment a1 = sys.schedule(d1, p1, "10:00 AM");
Appointment a2 = sys.schedule(d1, p2, "10:00 AM"); // Overlapping
} catch (Exception e) {
System.out.println("Error: " + e.getMessage());
}
try {
sys.schedule(d1, null, "11:00 AM"); // null patient
} catch (Exception e) {
System.out.println("Error: " + e.getMessage());
}
}
}
}
```

Expected Output

```
Booked: APPT1 | Dr.Sharma | Kunal | Slot: 10:00 AM | CONFIRMED
```


Error: Slot 10:00 AM already booked for Dr.Sharma

Error: Patient info cannot be null.

Question 10: Smart Home Device Control System

Package: com.smarthome.devices

Overview

- Base class: Device | Subclasses: Light, Fan | Interfaces: Switchable, Configurable
- Supports on/off toggle and device settings (brightness, speed).
- Handles: device offline errors, invalid configuration values.

OOP / Rubric Highlights

- Multiple interface implementation (Switchable, Configurable)
- final deviceId; static deviceCount
- Polymorphism: configure() overridden in Light and Fan

Complete Java Code

```
// File: Switchable.java
package com.smarthome.devices;

public interface Switchable {
    void turnOn() throws Exception;
    void turnOff();
    boolean isOn();
}

// File: Configurable.java
package com.smarthome.devices;

public interface Configurable {
    void configure(int value) throws Exception;
}

// File: Device.java
package com.smarthome.devices;

public abstract class Device implements Switchable {
    private static int deviceCount = 0;
    private final String deviceId;
    protected String name;
    protected boolean online;
    protected boolean powerOn;

    public Device(String name, boolean online) {
        this.deviceId = "DEV" + (++deviceCount);
        this.name = name;
        this.online = online;
        this.powerOn = false;
    }
}
```

```

@Override
public void turnOn() throws Exception {
    if (!online) throw new Exception(name + " is offline. Cannot turn on.");
    powerOn = true;
    System.out.println(name + " turned ON.");
}

@Override
public void turnOff() { powerOn = false; System.out.println(name + " turned OFF."); }

@Override
public boolean isOn() { return powerOn; }

public String getDeviceId() { return deviceId; }

@Override
public String toString() { return deviceId + " | " + name + " | " + (powerOn ? "ON" : "OFF"); }
}

```

```

// File: Light.java
package com.smarthome.devices;

public class Light extends Device implements Configurable {
    private int brightness; // 0-100%

```

```

    public Light(String name, boolean online) {
        super(name, online);
        this.brightness = 50;
    }

```

```

@Override
public void configure(int value) throws Exception {
    if (!isOn()) throw new Exception(name + " is off. Turn it on first.");
    if (value < 0 || value > 100)
        throw new IllegalArgumentException("Brightness must be 0-100.");
    this.brightness = value;
    System.out.println(name + " brightness set to " + brightness + "%");
}
}

```

```

// File: Fan.java
package com.smarthome.devices;

public class Fan extends Device implements Configurable {
    private int speed; // 1-5

    public Fan(String name, boolean online) {
        super(name, online);
        this.speed = 1;
    }

```

```

@Override
public void configure(int value) throws Exception {
    if (!isOn()) throw new Exception(name + " is off. Turn it on first.");
    if (value < 1 || value > 5)
        throw new IllegalArgumentException("Fan speed must be 1-5.");
    this.speed = value;
    System.out.println(name + " speed set to " + speed);
}
}

```

```

// File: Main.java
package com.smarthome.devices;

public class Main {
    public static void main(String[] args) {
        Light light = new Light("Living Room Light", true);
        Fan fan = new Fan("Bedroom Fan", false); // offline

        try {
            light.turnOn();
            light.configure(80);
            fan.turnOn(); // Should throw - offline
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }

        try {
            light.configure(150); // Invalid brightness
        } catch (Exception e) {
            System.out.println("Config Error: " + e.getMessage());
        }
        light.turnOff();
    }
}

```

Expected Output

```

Living Room Light turned ON.
Living Room Light brightness set to 80%
Error: Bedroom Fan is offline. Cannot turn on.
Config Error: Brightness must be 0-100.
Living Room Light turned OFF.

```

Question 11: Airline Ticketing System

Package: com.airline.booking

Overview

- Classes: Flight, Passenger, Ticket | Interface: CheckIn
- Polymorphism via overridden toString() and bookSeat().
- Handles: double booking, invalid passenger, overbooked flights.

OOP / Rubric Highlights

- Inheritance and polymorphism via Passenger subclasses (Business, Economy)
- static ticketCounter; final ticketId
- Interface CheckIn with checkIn() method

Complete Java Code

```
// File: CheckIn.java
package com.airline.booking;

public interface CheckIn {
    void checkIn(Ticket ticket) throws Exception;
}

// File: Flight.java
package com.airline.booking;
import java.util.HashSet;

public class Flight {
    private final String flightNumber;
    private String origin, destination;
    private int totalSeats;
    private int bookedSeats;
    private HashSet<String> bookedPassengerIds = new HashSet<>();
    private static final double BASE_FARE = 3500.0;

    public Flight(String flightNumber, String origin, String destination, int seats) {
        this.flightNumber = flightNumber;
        this.origin = origin;
        this.destination = destination;
        this.totalSeats = seats;
        this.bookedSeats = 0;
    }

    public void bookSeat(String passengerId) throws Exception {
        if (bookedSeats >= totalSeats)
            throw new Exception("Flight " + flightNumber + " is overbooked.");
        if (bookedPassengerIds.contains(passengerId))
```

```

throw new Exception("Passenger " + passengerId + " already booked.");
bookedPassengerIds.add(passengerId);
bookedSeats++;
}

public double getFare() { return BASE_FARE; }
public String getFlightNumber() { return flightNumber; }
public int getAvailableSeats() { return totalSeats - bookedSeats; }
@Override
public String toString() {
return flightNumber + " | " + origin + " -> " + destination +
" | Seats: " + getAvailableSeats() + "/" + totalSeats;
}
}

// File: Passenger.java
package com.airline.booking;
public class Passenger {
private static int pidCount = 0;
private final String passengerId;
private String name; private int age;

public Passenger(String name, int age) {
if (name == null || name.isBlank()) throw new IllegalArgumentException("Name required.");
if (age <= 0 || age > 120) throw new IllegalArgumentException("Invalid age.");
this.passengerId = "PAS" + (++pidCount);
this.name = name; this.age = age;
}

public String getPassengerId() { return passengerId; }
public String getName() { return name; }
@Override
public String toString() { return passengerId + " | " + name + " | Age: " + age; }
}

// File: Ticket.java
package com.airline.booking;
public class Ticket {
private static int tCount = 0;
private final String ticketId;
private Flight flight; private Passenger passenger;
private double fare; private boolean checkedIn;

public Ticket(Flight flight, Passenger passenger) throws Exception {
flight.bookSeat(passenger.getPassengerId());
this.ticketId = "AIR" + (++tCount);
this.flight = flight;
this.passenger = passenger;
}
}

```

```

this.fare = flight.getFare();
this.checkedIn = false;
}
public void doCheckIn() { checkedIn = true; System.out.println(ticketId + " checked in."); }
public String getTicketId() { return ticketId; }
@Override
public String toString() {
return ticketId + " | " + passenger.getName() + " | " + flight.getFlightNumber() +
" | Rs." + fare + " | CheckedIn:" + checkedIn;
}
}

// File: Main.java
package com.airline.booking;
public class Main {
public static void main(String[] args) {
Flight f1 = new Flight("AI202", "Mumbai", "Delhi", 2);
try {
Passenger p1 = new Passenger("Kunal", 21);
Passenger p2 = new Passenger("Priya", 25);
Passenger p3 = new Passenger("Ravi", 30);

Ticket t1 = new Ticket(f1, p1);
System.out.println("Booked: " + t1);
Ticket t2 = new Ticket(f1, p2);
System.out.println("Booked: " + t2);
t1.doCheckIn();
new Ticket(f1, p3); // Overbooked
} catch (Exception e) {
System.out.println("Error: " + e.getMessage());
}
}
}

```

Expected Output

```

Booked: AIR1 | Kunal | AI202 | Rs.3500.0 | CheckedIn:false
Booked: AIR2 | Priya | AI202 | Rs.3500.0 | CheckedIn:false
AIR1 checked in.
Error: Flight AI202 is overbooked.

```

Question 12: Online Quiz Management System

Package: com.quiz.app

Overview

- Classes: Question, User, Quiz | Interface: Scorable
- Presents multiple-choice questions, accepts answers, calculates score.
- Handles: invalid input format, unanswered questions, score overflow.

OOP / Rubric Highlights

- ArrayList<Question> in Quiz; Scorable interface with calculateScore()
- final questionId; static questionCounter
- Encapsulation: correct answer hidden, checked via validate()

Complete Java Code

```
// File: Scorable.java
package com.quiz.app;
public interface Scorable {
    int calculateScore();
    void displayResult();
}

// File: Question.java
package com.quiz.app;
public class Question {
    private static int qCount = 0;
    private final String questionId;
    private String questionText;
    private String[] options;
    private char correctOption; // 'A', 'B', 'C', 'D'

    public Question(String questionText, String[] options, char correctOption) {
        if (options.length != 4) throw new IllegalArgumentException("Exactly 4 options required.");
        char c = Character.toUpperCase(correctOption);
        if (c < 'A' || c > 'D') throw new IllegalArgumentException("Correct option must be A-D.");
        this.questionId = "Q" + (++qCount);
        this.questionText = questionText;
        this.options = options;
        this.correctOption = c;
    }

    public boolean validate(char userAnswer) {
        return Character.toUpperCase(userAnswer) == correctOption;
    }
}
```



```

}

public void display(int num) {
    System.out.println("\n" + num + ". " + questionText);
    char opt = 'A';
    for (String o : options) System.out.println(" " + (opt++) + ") " + o);
}

public String getId() { return questionId; }
}

// File: User.java
package com.quiz.app;
public class User {
    private String username;
    public User(String username) {
        if (username == null || username.isBlank())
            throw new IllegalArgumentException("Username cannot be empty.");
        this.username = username;
    }
    public String getUsername() { return username; }
}

// File: Quiz.java
package com.quiz.app;
import java.util.ArrayList;
public class Quiz implements Scorable {
    private static final int MAX_SCORE = 100;
    private String quizTitle;
    private ArrayList<Question> questions = new ArrayList<>();
    private ArrayList<Character> userAnswers = new ArrayList<>();
    private User user;

    public Quiz(String quizTitle, User user) {
        this.quizTitle = quizTitle;
        this.user = user;
    }

    public void addQuestion(Question q) { questions.add(q); }

    public void submitAnswer(char answer) {
        char a = Character.toUpperCase(answer);
        if (a < 'A' || a > 'D') throw new IllegalArgumentException("Answer must be A-D.");
        userAnswers.add(a);
    }

    @Override
    public int calculateScore() {

```

```

if (userAnswers.size() != questions.size())
throw new RuntimeException("All questions must be answered.");
int correct = 0;
for (int i = 0; i < questions.size(); i++)
if (questions.get(i).validate(userAnswers.get(i))) correct++;
int score = (correct * MAX_SCORE) / questions.size();
if (score > MAX_SCORE) throw new ArithmeticException("Score overflow.");
return score;
}

@Override
public void displayResult() {
int score = calculateScore();
System.out.println("\n=== QUIZ RESULT: " + quizTitle + " ===");
System.out.println("Student : " + user.getUsername());
System.out.println("Score : " + score + "/" + MAX_SCORE);
System.out.println("Grade : " + (score >= 90 ? "A" : score >= 75 ? "B" :
score >= 60 ? "C" : "F"));
}
}

```

```

// File: Main.java
package com.quiz.app;
public class Main {
public static void main(String[] args) {
User u = new User("Kunal");
Quiz q = new Quiz("Java OOP Quiz", u);

q.addQuestion(new Question("What is JVM?",
new String[]{"Java Virtual Machine", "Java Version Manager",
"Java Verified Module", "None"}, 'A'));
q.addQuestion(new Question("Which keyword prevents inheritance?",
new String[]{"static", "final", "abstract", "private"}, 'B'));
q.addQuestion(new Question("OOP pillar for hiding data:",
new String[]{"Inheritance", "Polymorphism", "Encapsulation", "Abstraction"}, 'C'));
q.addQuestion(new Question("ArrayList is in package:",
new String[]{"java.io", "java.lang", "java.util", "java.net"}, 'C'));

q.submitAnswer('A');
q.submitAnswer('B');
q.submitAnswer('C');
q.submitAnswer('C');

q.displayResult();
}
}

```

Expected Output

```
=== QUIZ RESULT: Java OOP Quiz ===
```

```
Student : Kunal
```

```
Score : 100/100
```

```
Grade : A
```

Question 13: Customer Feedback and Rating System

Package: com.feedback.system

Overview

- Classes: Customer, Feedback, Product | Interface: Reviewable
- Accepts 1-5 star ratings; calculates average rating per product.
- Handles: out-of-range ratings, duplicate feedback, null reviews.

OOP / Rubric Highlights

- ArrayList<Feedback> in Product; Reviewable interface
- final feedbackId; HashSet to track who already reviewed

Complete Java Code

```
// File: Reviewable.java
package com.feedback.system;
public interface Reviewable {
    void addFeedback(Customer c, int rating, String comment) throws Exception;
    double getAverageRating();
}

// File: Customer.java
package com.feedback.system;
public class Customer {
    private static int cCount = 0;
    private final String customerId;
    private String name;
    public Customer(String name) {
        if (name == null || name.isBlank()) throw new IllegalArgumentException("Name required.");
        this.customerId = "CUS" + (++cCount);
        this.name = name;
    }
    public String getCustomerId() { return customerId; }
    public String getName() { return name; }
}

// File: Feedback.java
package com.feedback.system;
public class Feedback {
    private static int fCount = 0;
    private final String feedbackId;
    private String customerName;
    private int rating;
    private String comment;
```

```

public Feedback(String customerName, int rating, String comment) {
    if (rating < 1 || rating > 5)
        throw new IllegalArgumentException("Rating must be 1-5.");
    if (comment == null) throw new NullPointerException("Comment cannot be null.");
    this.feedbackId = "FB" + (++fCount);
    this.customerName = customerName;
    this.rating = rating;
    this.comment = comment;
}

public int getRating() { return rating; }
@Override
public String toString() {
    return feedbackId + " | " + customerName + " | " + rating + " * | " + comment;
}
}

```

```

// File: Product.java
package com.feedback.system;
import java.util.*;

public class Product implements Reviewable {
    private String productName;
    private List<Feedback> feedbacks = new ArrayList<>();
    private Set<String> reviewerIds = new HashSet<>();

    public Product(String productName) { this.productName = productName; }

    @Override
    public void addFeedback(Customer c, int rating, String comment) throws Exception {
        if (reviewerIds.contains(c.getCustomerId()))
            throw new Exception("Duplicate feedback from " + c.getName());
        feedbacks.add(new Feedback(c.getName(), rating, comment));
        reviewerIds.add(c.getCustomerId());
        System.out.println("Feedback added by " + c.getName());
    }

    @Override
    public double getAverageRating() {
        if (feedbacks.isEmpty()) return 0.0;
        return feedbacks.stream().mapToInt(Feedback::getRating).average().orElse(0.0);
    }

    public void printSummary() {
        System.out.println("\nProduct: " + productName);
        feedbacks.forEach(System.out::println);
        System.out.printf("Average Rating: %.2f / 5\n", getAverageRating());
    }
}

```

```

}

// File: Main.java
package com.feedback.system;

public class Main {
    public static void main(String[] args) {
        Product p = new Product("Wireless Headphones");
        Customer c1 = new Customer("Kunal");
        Customer c2 = new Customer("Priya");

        try {
            p.addFeedback(c1, 5, "Excellent sound quality!");
            p.addFeedback(c2, 4, "Good bass, slightly heavy.");
            p.addFeedback(c1, 3, "Changed my mind"); // Duplicate – throws
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }

        try {
            p.addFeedback(new Customer("Ravi"), 7, "Great"); // Invalid rating
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }

        p.printSummary();
    }
}

```

Expected Output

```

Feedback added by Kunal
Feedback added by Priya
Error: Duplicate feedback from Kunal
Error: Rating must be 1-5.

Product: Wireless Headphones
FB1 | Kunal | 5* | Excellent sound quality!
FB2 | Priya | 4* | Good bass, slightly heavy.
Average Rating: 4.50 / 5

```

Question 14: Food Delivery App Simulation

Package: com.fooddelivery.app

Overview

- Classes: Restaurant, Order, DeliveryPerson | Interface: Deliverable
- Uses composition: Order contains Restaurant and DeliveryPerson references.
- Handles: order delays, unavailable delivery personnel, invalid order format.

OOP / Rubric Highlights

- Composition over inheritance: Order HAS-A Restaurant and HAS-A DeliveryPerson
- Interface Deliverable with deliver() method
- static orderCounter; final orderId

Complete Java Code

```
// File: Deliverable.java
package com.fooddelivery.app;
public interface Deliverable {
    void deliver() throws Exception;
    String getStatus();
}

// File: Restaurant.java
package com.fooddelivery.app;
public class Restaurant {
    private String name, cuisine;
    private boolean isOpen;
    public Restaurant(String name, String cuisine, boolean isOpen) {
        this.name = name; this.cuisine = cuisine; this.isOpen = isOpen;
    }
    public boolean isOpen() { return isOpen; }
    public String getName() { return name; }
    @Override
    public String toString() { return name + " (" + cuisine + ")"; }
}

// File: DeliveryPerson.java
package com.fooddelivery.app;
public class DeliveryPerson {
    private String name;
    private boolean available;
    public DeliveryPerson(String name) { this.name = name; this.available = true; }
    public boolean isAvailable() { return available; }
    public void assignOrder() { available = false; }
    public void completeOrder() { available = true; }
```

```

public String getName() { return name; }
}

// File: Order.java
package com.fooddelivery.app;
public class Order implements Deliverable {
private static int oCount = 0;
private final String orderId;
private Restaurant restaurant;
private DeliveryPerson deliveryPerson;
private String foodItem;
private double totalAmount;
private String status;

public Order(Restaurant r, DeliveryPerson dp, String foodItem, double amount) throws
Exception {
if (!r.isOpen()) throw new Exception(r.getName() + " is currently closed.");
if (!dp.isAvailable()) throw new Exception(dp.getName() + " is unavailable.");
if (foodItem == null || foodItem.isBlank())
throw new IllegalArgumentException("Invalid food item.");
if (amount <= 0) throw new IllegalArgumentException("Order amount must be positive.");
this.orderId = "ORD" + (++oCount);
this.restaurant = r;
this.deliveryPerson = dp;
this.foodItem = foodItem;
this.totalAmount = amount;
this.status = "PLACED";
dp.assignOrder();
}

@Override
public void deliver() throws Exception {
status = "OUT_FOR_DELIVERY";
System.out.println(orderId + ": Out for delivery by " + deliveryPerson.getName());
status = "DELIVERED";
deliveryPerson.completeOrder();
System.out.println(orderId + ": Delivered! Amount: Rs." + totalAmount);
}

@Override
public String getStatus() { return status; }

@Override
public String toString() {
return orderId + " | " + foodItem + " | " + restaurant.getName() +
" | Rs." + totalAmount + " | " + status;
}
}

```



```

}

// File: Main.java
package com.fooddelivery.app;

public class Main {
    public static void main(String[] args) {
        Restaurant r1 = new Restaurant("Pizza Palace", "Italian", true);
        Restaurant r2 = new Restaurant("Dosa Hub", "South Indian", false);
        DeliveryPerson dp = new DeliveryPerson("Rahul");
        try {
            Order o1 = new Order(r1, dp, "Margherita Pizza", 349.0);
            System.out.println("Placed: " + o1);
            o1.deliver();

            Order o2 = new Order(r2, dp, "Masala Dosa", 129.0); // Closed restaurant
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}

```

Expected Output

```

Placed: ORD1 | Margherita Pizza | Pizza Palace | Rs.349.0 | PLACED
ORD1: Out for delivery by Rahul
ORD1: Delivered! Amount: Rs.349.0
Error: Dosa Hub is currently closed.

```

Question 15: Parcel Tracking System

Package: com.parcel.tracking

Overview

- Classes: Parcel, Location, Courier | Interface: Trackable
- Tracks parcel status updates and predicts delivery time.
- Handles: missing location updates, invalid tracking numbers.

OOP / Rubric Highlights

- ArrayList<Location> for status history
- final trackingNumber; static parcelCount
- Interface Trackable with updateStatus() and getETA()

Complete Java Code

```
// File: Trackable.java
package com.parcel.tracking;

public interface Trackable {
    void updateStatus(String location, String status);
    String getETA();
    void printHistory();
}

// File: Location.java
package com.parcel.tracking;

public class Location {
    private String place;
    private String status;
    private String timestamp;

    public Location(String place, String status) {
        this.place = place;
        this.status = status;
        this.timestamp = java.time.LocalDateTime.now()
            .format(java.time.format.DateTimeFormatter.ofPattern("dd-MM HH:mm"));
    }

    @Override
    public String toString() {
        return timestamp + " | " + place + " | " + status;
    }

    public String getStatus() { return status; }
}

// File: Parcel.java
package com.parcel.tracking;
```

```

import java.util.ArrayList;

public class Parcel implements Trackable {
    private static int pCount = 0;
    private final String trackingNumber;
    private String senderName, receiverName;
    private ArrayList<Location> history = new ArrayList<>();

    public Parcel(String senderName, String receiverName) {
        if (senderName == null || receiverName == null)
            throw new NullPointerException("Sender/Receiver info required.");
        this.trackingNumber = "TRK" + String.format("%06d", ++pCount);
        this.senderName = senderName;
        this.receiverName = receiverName;
        history.add(new Location("Origin Warehouse", "PICKED_UP"));
    }

    @Override
    public void updateStatus(String location, String status) {
        if (location == null || status == null)
            throw new IllegalArgumentException("Location and status cannot be null.");
        history.add(new Location(location, status));
        System.out.println("Status updated: " + location + " - " + status);
    }

    @Override
    public String getETA() {
        String lastStatus = history.get(history.size() - 1).getStatus();
        return switch (lastStatus) {
            case "PICKED_UP" -> "3-5 business days";
            case "IN_TRANSIT" -> "1-2 business days";
            case "OUT_FOR_DELIVERY" -> "Today";
            case "DELIVERED" -> "Delivered";
            default -> "Unknown";
        };
    }

    @Override
    public void printHistory() {
        System.out.println("\nTracking: " + trackingNumber +
            " | " + senderName + " -> " + receiverName);
        history.forEach(System.out::println);
        System.out.println("ETA: " + getETA());
    }

    public String getTrackingNumber() { return trackingNumber; }
}

```

```
// File: Main.java
package com.parcel.tracking;

public class Main {
    public static void main(String[] args) {
        Parcel p = new Parcel("Kunal", "Priya");
        System.out.println("Tracking Number: " + p.getTrackingNumber());
        p.updateStatus("Pune Sorting Hub", "IN_TRANSIT");
        p.updateStatus("Mumbai Hub", "IN_TRANSIT");
        p.updateStatus("Delivery Agent", "OUT_FOR_DELIVERY");
        p.printHistory();

        try {
            p.updateStatus(null, "DELIVERED");
        } catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Expected Output

```
Tracking Number: TRK000001
Status updated: Pune Sorting Hub - IN_TRANSIT
Status updated: Mumbai Hub - IN_TRANSIT
Status updated: Delivery Agent - OUT_FOR_DELIVERY

Tracking: TRK000001 | Kunal -> Priya
... [timestamp] | Origin Warehouse | PICKED_UP
... [timestamp] | Pune Sorting Hub | IN_TRANSIT
... [timestamp] | Delivery Agent | OUT_FOR_DELIVERY
ETA: Today
Error: Location and status cannot be null.
```

Question 16: Electricity Bill Generator

Package: com.utility.billing

Overview

- Classes: Customer, Bill, Meter (with inheritance) | Interface: Billable
- Slab-based billing: 0-100 units @ Rs.2, 101-300 @ Rs.5, 300+ @ Rs.8
- Handles: invalid meter readings, negative units, slab mismatch.

OOP / Rubric Highlights

- Inheritance: Meter as base; CommercialMeter extends Meter with surcharge
- final meterId, customerId; static constants for slab rates
- Billable interface with generateBill() method

Complete Java Code

```
// File: Billable.java
package com.utility.billing;
public interface Billable {
    double calculateAmount(int units);
    void generateBill();
}

// File: Meter.java
package com.utility.billing;
public class Meter {
    private static int mCount = 0;
    private final String meterId;
    protected int previousReading;
    protected int currentReading;

    public Meter(int previousReading, int currentReading) {
        if (previousReading < 0 || currentReading < 0)
            throw new IllegalArgumentException("Meter readings cannot be negative.");
        if (currentReading < previousReading)
            throw new IllegalArgumentException("Current reading cannot be less than previous.");
        this.meterId = "MTR" + (++mCount);
        this.previousReading = previousReading;
        this.currentReading = currentReading;
    }

    public int getUnitsConsumed() { return currentReading - previousReading; }
    public String getMeterId() { return meterId; }
}

// File: Bill.java
```

```

package com.utility.billing;

public class Bill implements Billable {
    private static final double RATE_SLAB1 = 2.0; // 0-100 units
    private static final double RATE_SLAB2 = 5.0; // 101-300 units
    private static final double RATE_SLAB3 = 8.0; // 300+ units
    private static final double FIXED_CHARGE = 50.0;

    private Meter meter;
    private String customerName;
    private String billMonth;

    public Bill(Meter meter, String customerName, String billMonth) {
        this.meter = meter;
        this.customerName = customerName;
        this.billMonth = billMonth;
    }

    @Override
    public double calculateAmount(int units) {
        if (units < 0) throw new IllegalArgumentException("Units cannot be negative.");
        double amount = 0;
        if (units <= 100) {
            amount = units * RATE_SLAB1;
        } else if (units <= 300) {
            amount = 100 * RATE_SLAB1 + (units - 100) * RATE_SLAB2;
        } else {
            amount = 100 * RATE_SLAB1 + 200 * RATE_SLAB2 + (units - 300) * RATE_SLAB3;
        }
        return amount + FIXED_CHARGE;
    }

    @Override
    public void generateBill() {
        int units = meter.getUnitsConsumed();
        double total = calculateAmount(units);
        System.out.println("\n===== ELECTRICITY BILL =====");
        System.out.println("Customer : " + customerName);
        System.out.println("Month : " + billMonth);
        System.out.println("Meter ID : " + meter.getMeterId());
        System.out.println("Units : " + units);
        System.out.printf ("Fixed : Rs.%.2f\n", FIXED_CHARGE);
        System.out.printf ("Total : Rs.%.2f\n", total);
        System.out.println("=====");
    }
}

// File: Main.java

```

```

package com.utility.billing;

public class Main {
    public static void main(String[] args) {
        try {
            Meter m1 = new Meter(1200, 1580); // 380 units
            Bill bill1 = new Bill(m1, "Kunal Sharma", "May 2025");
            bill1.generateBill();

            Meter m2 = new Meter(500, 560); // 60 units
            Bill bill2 = new Bill(m2, "Priya Patil", "May 2025");
            bill2.generateBill();

            Meter bad = new Meter(600, 500); // Invalid: current < previous
        } catch (IllegalArgumentException e) {
            System.out.println("Meter Error: " + e.getMessage());
        }
    }
}

```

Expected Output

```

===== ELECTRICITY BILL =====
Customer : Kunal Sharma
Month : May 2025
Meter ID : MTR1
Units : 380
Fixed : Rs.50.00
Total : Rs.1090.00
=====

===== ELECTRICITY BILL =====
Customer : Priya Patil
Month : May 2025
Meter ID : MTR2
Units : 60
Fixed : Rs.50.00
Total : Rs.170.00
=====

Meter Error: Current reading cannot be less than previous.

```

Question 17: Event Management System

Package: com.events.manager

Overview

- Classes: Event, Venue, Attendee | Interface: Schedulable
- Manages bookings, available slot tracking, capacity enforcement.
- Handles: overbooking, double bookings, invalid event dates.

OOP / Rubric Highlights

- ArrayList<Attendee> in Event; HashSet for duplicate check
- final eventId; static eventCount
- Schedulable interface shared with Hospital (Q9) pattern

Complete Java Code

```
// File: Schedulable.java
package com.events.manager;

public interface Schedulable {
    void registerAttendee(Attendee a) throws Exception;
    void cancelRegistration(Attendee a);
    int getAvailableSlots();
}

// File: Venue.java
package com.events.manager;

public class Venue {
    private String venueName;
    private int capacity;

    public Venue(String venueName, int capacity) {
        if (capacity <= 0) throw new IllegalArgumentException("Capacity must be positive.");
        this.venueName = venueName;
        this.capacity = capacity;
    }

    public int getCapacity() { return capacity; }
    public String getVenueName() { return venueName; }
}

// File: Attendee.java
package com.events.manager;

public class Attendee {
    private static int aCount = 0;
    private final String attendeeId;
    private String name;

    public Attendee(String name) {
        if (name == null || name.isBlank()) throw new IllegalArgumentException("Name required.");
```



```

this.attendeeId = "ATT" + (++aCount);
this.name = name;
}
public String getAttendeeId() { return attendeeId; }
public String getName() { return name; }
}

// File: Event.java
package com.events.manager;
import java.util.*;
public class Event implements Schedulable {
private static int eCount = 0;
private final String eventId;
private String eventName, eventDate;
private Venue venue;
private List<Attendee> attendees = new ArrayList<>();
private Set<String> registered = new HashSet<>();

public Event(String eventName, String eventDate, Venue venue) {
if (eventName == null || eventDate == null)
throw new IllegalArgumentException("Event name/date cannot be null.");
this.eventId = "EVT" + (++eCount);
this.eventName = eventName;
this.eventDate = eventDate;
this.venue = venue;
}

@Override
public void registerAttendee(Attendee a) throws Exception {
if (attendees.size() >= venue.getCapacity())
throw new Exception("Event is at full capacity. Overbooking prevented.");
if (registered.contains(a.getAttendeeId()))
throw new Exception(a.getName() + " is already registered.");
attendees.add(a);
registered.add(a.getAttendeeId());
System.out.println(a.getName() + " registered for " + eventName);
}

@Override
public void cancelRegistration(Attendee a) {
attendees.removeIf(x -> x.getAttendeeId().equals(a.getAttendeeId()));
registered.remove(a.getAttendeeId());
System.out.println(a.getName() + "'s registration cancelled.");
}

@Override
public int getAvailableSlots() { return venue.getCapacity() - attendees.size(); }

```

```

public void printDetails() {
    System.out.println("\n" + eventId + " | " + eventName + " | " + eventDate +
        " | Venue: " + venue.getVenueName() +
        " | Slots Left: " + getAvailableSlots());
}
}

```

```

// File: Main.java
package com.events.manager;

public class Main {
    public static void main(String[] args) {
        Venue v = new Venue("MITAOE Auditorium", 2);
        Event e = new Event("Tech Summit 2025", "20-06-2025", v);
        Attendee a1 = new Attendee("Kunal");
        Attendee a2 = new Attendee("Priya");
        Attendee a3 = new Attendee("Ravi");
        try {
            e.registerAttendee(a1);
            e.registerAttendee(a2);
            e.registerAttendee(a1); // Duplicate
        } catch (Exception ex) {
            System.out.println("Error: " + ex.getMessage());
        }
        try {
            e.registerAttendee(a3); // Full capacity
        } catch (Exception ex) {
            System.out.println("Error: " + ex.getMessage());
        }
        e.printDetails();
    }
}

```

Expected Output

```

Kunal registered for Tech Summit 2025
Priya registered for Tech Summit 2025
Error: Kunal is already registered.
Error: Event is at full capacity. Overbooking prevented.

EVT1 | Tech Summit 2025 | 20-06-2025 | Venue: MITAOE Auditorium | Slots Left: 0

```

Question 18: Hotel Room Booking System

Package: com.hotel.booking

Overview

- Classes: Room, Customer, Booking | Interface: Bookable
- Supports room type selection, availability check, pricing.
- Handles: no rooms available, invalid customer info, payment failures.

OOP / Rubric Highlights

- Enum RoomType: STANDARD, DELUXE, SUITE with different rates
- static bookingCounter; final bookingId
- Bookable interface with book() and checkout() methods

Complete Java Code

```
// File: Bookable.java
package com.hotel.booking;

public interface Bookable {
    Booking book(Customer c, int nights) throws Exception;
    void checkout(Booking b);
}
```

```
// File: Room.java
package com.hotel.booking;

public class Room implements Bookable {
    public enum RoomType { STANDARD, DELUXE, SUITE }
    private static int rCount = 0;
    private final int roomNumber;
    private RoomType type;
    private boolean isAvailable;
    private final double pricePerNight;

    public Room(RoomType type) {
        this.roomNumber = 100 + (++rCount);
        this.type = type;
        this.isAvailable = true;
        this.pricePerNight = switch (type) {
            case STANDARD -> 2000.0;
            case DELUXE -> 4000.0;
            case SUITE -> 8000.0;
        };
    }

    @Override
```

```

public Booking book(Customer c, int nights) throws Exception {
    if (!isAvailable) throw new Exception("Room " + roomNumber + " is already booked.");
    if (nights <= 0) throw new IllegalArgumentException("Nights must be positive.");
    isAvailable = false;
    Booking b = new Booking(this, c, nights);
    System.out.println("Room " + roomNumber + " (" + type + ") booked for " + c.getName());
    return b;
}

@Override
public void checkout(Booking b) {
    isAvailable = true;
    System.out.printf("Checkout: Room %d | Total: Rs.%.2f%n",
        roomNumber, b.getTotalAmount());
}

public int getRoomNumber() { return roomNumber; }
public boolean isAvailable() { return isAvailable; }
public double getPricePerNight() { return pricePerNight; }
public RoomType getType() { return type; }
}

```

```

// File: Customer.java
package com.hotel.booking;

public class Customer {
    private static int cCount = 0;
    private final String customerId;
    private String name, phone;

    public Customer(String name, String phone) {
        if (name == null || name.isBlank())
            throw new IllegalArgumentException("Customer name required.");
        if (phone == null || phone.length() != 10)
            throw new IllegalArgumentException("Invalid phone number.");
        this.customerId = "HOT" + (++cCount);
        this.name = name;
        this.phone = phone;
    }

    public String getName() { return name; }
    public String getCustomerId() { return customerId; }
}

```

```

// File: Booking.java
package com.hotel.booking;

public class Booking {
    private static int bCount = 0;
    private final String bookingId;

```

```

private Room room; private Customer customer;
private int nights; private double totalAmount;

public Booking(Room room, Customer customer, int nights) {
this.bookingId = "BKG" + (++bCount);
this.room = room;
this.customer = customer;
this.nights = nights;
this.totalAmount = room.getPricePerNight() * nights;
}
public double getTotalAmount() { return totalAmount; }
public String getBookingId() { return bookingId; }
@Override
public String toString() {
return bookingId + " | " + customer.getName() + " | Room:" +
room.getRoomNumber() + " " + room.getType() +
" | " + nights + " nights | Rs." + totalAmount;
}
}

```

```

// File: Main.java
package com.hotel.booking;
public class Main {
public static void main(String[] args) {
Room r1 = new Room(Room.RoomType.DELUXE);
Room r2 = new Room(Room.RoomType.SUITE);

try {
Customer c1 = new Customer("Kunal", "9876543210");
Customer c2 = new Customer("Priya", "9123456789");
Booking b1 = r1.book(c1, 3);
System.out.println(b1);
Booking b2 = r1.book(c2, 2); // Room not available
} catch (Exception e) {
System.out.println("Error: " + e.getMessage());
}

try {
Customer bad = new Customer("X", "123"); // Invalid phone
} catch (IllegalArgumentException e) {
System.out.println("Customer Error: " + e.getMessage());
}
}
}
}

```

Expected Output

```

Room 101 (DELUXE) booked for Kunal
BKG1 | Kunal | Room:101 DELUXE | 3 nights | Rs.12000.0

```

Error: Room 101 is already booked.

Customer Error: Invalid phone number.

Question 19: Loan Eligibility Checker

Package: com.finance.loans

Overview

- Classes: Loan, Applicant, CreditScore | Interface: Eligible
- Checks loan eligibility based on age (21-60), income (≥ 25000), and credit score (≥ 650).
- Handles: incomplete applicant data, invalid scores, rule violations.

OOP / Rubric Highlights

- Eligible interface with checkEligibility() returning boolean
- CreditScore encapsulates score with validation
- final loanId; static counter; this() chaining in Loan

Complete Java Code

```
// File: Eligible.java
package com.finance.loans;
public interface Eligible {
    boolean checkEligibility();
    void displayEligibilityReport();
}

// File: CreditScore.java
package com.finance.loans;
public class CreditScore {
    private int score;
    private static final int MIN_VALID = 300;
    private static final int MAX_VALID = 900;

    public CreditScore(int score) {
        if (score < MIN_VALID || score > MAX_VALID)
            throw new IllegalArgumentException(
                "Credit score must be " + MIN_VALID + "-" + MAX_VALID + ".");
        this.score = score;
    }

    public int getScore() { return score; }
    public String getGrade() {
        if (score >= 750) return "Excellent";
        if (score >= 650) return "Good";
        if (score >= 550) return "Fair";
        return "Poor";
    }
}

// File: Applicant.java
```

```

package com.finance.loans;

public class Applicant {
    private static int aCount = 0;
    private final String applicantId;
    private String name;
    private int age;
    private double monthlyIncome;
    private CreditScore creditScore;

    public Applicant(String name, int age, double monthlyIncome, int creditScoreVal) {
        if (name == null || name.isBlank()) throw new IllegalArgumentException("Name required.");
        if (age <= 0) throw new IllegalArgumentException("Invalid age.");
        if (monthlyIncome <= 0) throw new IllegalArgumentException("Invalid income.");
        this.applicantId = "APP" + (++aCount);
        this.name = name;
        this.age = age;
        this.monthlyIncome = monthlyIncome;
        this.creditScore = new CreditScore(creditScoreVal);
    }

    public String getName() { return name; }
    public int getAge() { return age; }
    public double getMonthlyIncome() { return monthlyIncome; }
    public CreditScore getCreditScore(){ return creditScore; }
}

// File: Loan.java
package com.finance.loans;

public class Loan implements Eligible {
    private static int lCount = 0;
    private final String loanId;
    private Applicant applicant;
    private double loanAmount;
    private String loanType;

    private static final int MIN_AGE = 21;
    private static final int MAX_AGE = 60;
    private static final double MIN_INCOME = 25000.0;
    private static final int MIN_SCORE = 650;

    public Loan(Applicant applicant, double loanAmount, String loanType) {
        if (loanAmount <= 0) throw new IllegalArgumentException("Loan amount must be positive.");
        this.loanId = "LN" + (++lCount);
        this.applicant = applicant;
        this.loanAmount = loanAmount;
        this.loanType = loanType;
    }
}

```



```

}

@Override
public boolean checkEligibility() {
    int age = applicant.getAge();
    double income = applicant.getMonthlyIncome();
    int score = applicant.getCreditScore().getScore();
    return age >= MIN_AGE && age <= MAX_AGE &&
    income >= MIN_INCOME && score >= MIN_SCORE;
}

@Override
public void displayEligibilityReport() {
    boolean eligible = checkEligibility();
    System.out.println("\n=== LOAN ELIGIBILITY REPORT ===");
    System.out.println("Loan ID : " + loanId);
    System.out.println("Applicant : " + applicant.getName());
    System.out.println("Loan Type : " + loanType);
    System.out.printf ("Amount : Rs.%.2f%n", loanAmount);
    System.out.println("Age : " + applicant.getAge() +
    (applicant.getAge() >= MIN_AGE && applicant.getAge() <= MAX_AGE
    ? " ✓" : " ✗"));
    System.out.printf ("Income : Rs.%.0f%n", applicant.getMonthlyIncome(),
    applicant.getMonthlyIncome() >= MIN_INCOME ? " ✓" : " ✗");
    System.out.println("Credit : " + applicant.getCreditScore().getScore() +
    " (" + applicant.getCreditScore().getGrade() + ")" +
    (applicant.getCreditScore().getScore() >= MIN_SCORE ? " ✓" : " ✗"));
    System.out.println("RESULT : " + (eligible ? "APPROVED ✓" : "REJECTED ✗"));
}
}

// File: Main.java
package com.finance.loans;

public class Main {
    public static void main(String[] args) {
        try {
            Applicant a1 = new Applicant("Kunal", 25, 45000, 720);
            Applicant a2 = new Applicant("Priya", 19, 30000, 600);
            Applicant a3 = new Applicant("Ravi", 35, 20000, 700);

            new Loan(a1, 500000, "Home Loan").displayEligibilityReport();
            new Loan(a2, 200000, "Car Loan").displayEligibilityReport();
            new Loan(a3, 100000, "Personal Loan").displayEligibilityReport();

            // Invalid credit score
            new Applicant("Bad", 30, 30000, 200);
        } catch (IllegalArgumentException e) {

```

```
System.out.println("\nError: " + e.getMessage());  
}  
}  
}
```

Expected Output

```
=== LOAN ELIGIBILITY REPORT ===
```

```
Loan ID : LN1
```

```
Applicant : Kunal
```

```
Loan Type : Home Loan
```

```
Amount : Rs.500000.00
```

```
Age : 25 ✓
```

```
Income : Rs.45000 ✓
```

```
Credit : 720 (Excellent) ✓
```

```
RESULT : APPROVED ✓
```

```
=== LOAN ELIGIBILITY REPORT ===
```

```
RESULT : REJECTED ✗ (age < 21)
```

```
Error: Credit score must be 300-900.
```

Question 20: Weather Forecasting Application

Package: com.weather.app

Overview

- Classes: WeatherData, City, Forecast | Interface: Forecastable
- Calculates temperature average and humidity trends from data arrays.
- Handles: missing data entries, invalid temperature formats, null references.

OOP / Rubric Highlights

- Arrays used to store 7-day temperature and humidity data
- Forecastable interface with generateForecast() and getAverageTemp()
- final cityName; static dataCount for tracking entries

Complete Java Code

```
// File: Forecastable.java
package com.weather.app;
public interface Forecastable {
    double getAverageTemperature();
    double getAverageHumidity();
    void generateForecast();
}

// File: WeatherData.java
package com.weather.app;
public class WeatherData {
    private double temperature; // Celsius
    private double humidity; // percentage
    private String date;

    public WeatherData(String date, double temperature, double humidity) {
        if (date == null || date.isBlank())
            throw new NullPointerException("Date cannot be null.");
        if (temperature < -90 || temperature > 60)
            throw new IllegalArgumentException("Invalid temperature: " + temperature);
        if (humidity < 0 || humidity > 100)
            throw new IllegalArgumentException("Humidity must be 0-100.");
        this.date = date;
        this.temperature = temperature;
        this.humidity = humidity;
    }

    public double getTemperature() { return temperature; }
    public double getHumidity() { return humidity; }
    public String getDate() { return date; }
```

```

@Override
public String toString() {
return String.format("%s | Temp: %.1f°C | Humidity: %.1f%%", date, temperature, humidity);
}
}

// File: City.java
package com.weather.app;
public class City {
private String cityName;
private String state;
public City(String cityName, String state) {
if (cityName == null) throw new NullPointerException("City name required.");
this.cityName = cityName;
this.state = state;
}
public String getCityName() { return cityName; }
@Override
public String toString() { return cityName + ", " + state; }
}

// File: Forecast.java
package com.weather.app;
import java.util.ArrayList;
public class Forecast implements Forecastable {
private City city;
private ArrayList<WeatherData> dataList = new ArrayList<>();

public Forecast(City city) {
if (city == null) throw new NullPointerException("City reference cannot be null.");
this.city = city;
}

public void addData(WeatherData wd) {
if (wd == null) throw new NullPointerException("WeatherData cannot be null.");
dataList.add(wd);
}

@Override
public double getAverageTemperature() {
if (dataList.isEmpty()) throw new RuntimeException("No weather data available.");
return dataList.stream().mapToDouble(WeatherData::getTemperature).average().orElse(0.0);
}

@Override
public double getAverageHumidity() {
if (dataList.isEmpty()) throw new RuntimeException("No weather data available.");
}
}

```

```

return dataList.stream().mapToDouble(WeatherData::getHumidity).average().orElse(0.0);
}

@Override
public void generateForecast() {
    System.out.println("\n===== WEATHER FORECAST: " + city + " =====");
    dataList.forEach(System.out::println);
    System.out.printf("Avg Temperature : %.2f°C\n", getAverageTemperature());
    System.out.printf("Avg Humidity : %.2f%%\n", getAverageHumidity());
    double avg = getAverageTemperature();
    String condition = avg >= 35 ? "Hot" : avg >= 25 ? "Warm" :
    avg <= 15 ? "Pleasant" : "Cold";
    System.out.println("Forecast : " + condition);
}
}

// File: Main.java
package com.weather.app;
public class Main {
    public static void main(String[] args) {
        City pune = new City("Pune", "Maharashtra");
        Forecast fc = new Forecast(pune);

        String[] dates = {"Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"};
        double[] temps = {32.5, 34.0, 35.5, 36.0, 33.5, 31.0, 30.0};
        double[] humid = {65, 70, 75, 72, 68, 60, 58};

        try {
            for (int i = 0; i < 7; i++)
                fc.addData(new WeatherData(dates[i], temps[i], humid[i]));
            fc.generateForecast();
        } catch (Exception e) {
            System.out.println("\nError: " + e.getMessage());
        }
    }
}

```

Expected Output

```

===== WEATHER FORECAST: Pune, Maharashtra =====
Mon | Temp: 32.5°C | Humidity: 65.0%
Tue | Temp: 34.0°C | Humidity: 70.0%
...
Sun | Temp: 30.0°C | Humidity: 58.0%
Avg Temperature : 33.21°C

```

Avg Humidity : 66.86%

Forecast : Hot

Error: Invalid temperature: -100.0
